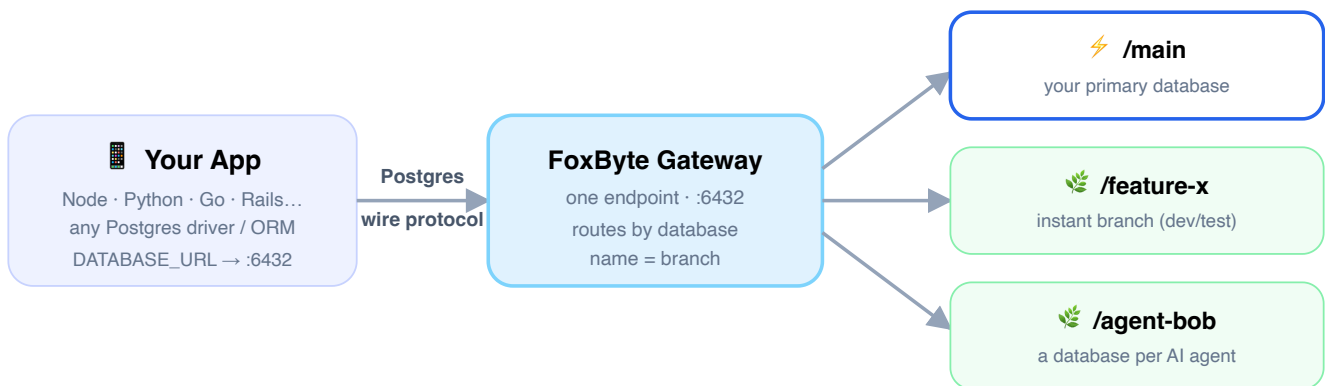


Using FoxByte in your application

FoxByte is **PostgreSQL** with three superpowers — instant branches, time-travel, and per-agent databases. Because it speaks the native Postgres wire protocol, **your existing driver, ORM, and SQL work unchanged**. This guide takes you from a clean machine to a running app doing full CRUD.

The one idea to hold onto: FoxByte gives your app a single, stable Postgres endpoint — `localhost:6432` — and the **database name you connect to is the branch name**. Connect to `/main` for your primary data, or `/feature-x` for an instant, isolated copy. Everything else is ordinary PostgreSQL.

How your application talks to FoxByte



Why this matters for you: you never manage per-branch ports or hosts. One connection string, and you swap the database name to point your app at production-like data or at a throwaway copy — instantly.

FoxByte — serverless PostgreSQL · branches · time-travel · agent databases. Open source (AGPL-3.0 core, Apache-2.0 clients). Full docs at the in-app **Docs** page.

1 Run FoxByte on your machine

Install the `fox` command; it sets up everything else (the local Linux VM on macOS, Docker, ZFS, the storage pool, and the image) for you.

macOS

```
# needs Lima for the local VM
brew install lima
curl -fsSL https://raw.githubusercontent.com/\
foxbyte/foxbyte/main/deploy/install.sh | sh
fox setup
```

Linux

```
curl -fsSL https://raw.githubusercontent.com/\
foxbyte/foxbyte/main/deploy/install.sh | sh
sudo fox start
```

`fox setup` (macOS) / `fox start` (Linux) brings up the database, the single gateway endpoint, and the APIs. Check it:

```
fox status      # servers + main + branches
fox stop        # stop everything
```

You now have:

WHAT	WHERE
SQL endpoint (your app connects here)	<code>localhost:6432</code>
Web console & dashboard	<code>make web-dev</code> → <code>localhost:5173</code>
Control API (REST)	<code>localhost:8080/api</code>
Agent API (a DB per agent)	<code>localhost:8088</code>

2 Create a branch and your schema

Start on `main`, or make an instant branch to work in isolation. Either is a normal PostgreSQL database — create your schema with plain SQL or your migration tool.

Optional: an isolated branch to build in

```
fox branch create dev      # instant copy-on-write branch of main
fox branch list            # see branches + their copy-on-write size
```

Create the schema (psql through the gateway)

```
psql "postgres://dbadmin:<API_KEY>@localhost:6432/dev" # or /main
```

```
CREATE TABLE notes (  
  id          serial PRIMARY KEY,  
  title       text NOT NULL,  
  body        text,  
  created_at  timestampz DEFAULT now()  
);
```

Prefer a GUI or your ORM's migrations? Point them at the same `postgres://dbadmin:<API_KEY>@localhost:6432/<branch>` URL — FoxByte is just PostgreSQL, so tools like Prisma Migrate, Alembic, golang-migrate, Flyway, and TablePlus all work.

Connection recap: host `localhost` · port `6432` · user `foxbyte` · password = your API key · database = branch name .
Create a key on the **API keys** page (or `fox apikey create <email>`).

3 Connect from your application

Put the connection string in an environment variable and use your language's standard Postgres library. Nothing FoxByte-specific is required in your code.

```
# .env
DATABASE_URL=postgres://dbadmin:<API_KEY>@localhost:6432/dev
```

Why `postgres://` ? Because FoxByte *is* PostgreSQL — the scheme tells your driver to speak the standard protocol, so every Postgres client and ORM connects with no changes.

The password is your API key. The Gateway (:6432) requires a valid key — create one on the **API keys** page (or `fox apikey create <email>`) and use it as the password.

Node.js — `pg`

```
import { Pool } from 'pg'
const pool = new Pool({ connectionString: process.env.DATABASE_URL })

const { rows } = await pool.query('SELECT now()')
console.log(rows[0])
```

Python — `psycopg (v3)`

```
import os, psycopg
with psycopg.connect(os.environ["DATABASE_URL"]) as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT now()")
        print(cur.fetchone())
```

Go — `pgx`

```
conn, err := pgx.Connect(ctx, os.Getenv("DATABASE_URL"))
if err != nil { log.Fatal(err) }
defer conn.Close(ctx)
```

ORMs & frameworks: Prisma, Drizzle, TypeORM, SQLAlchemy, Django ORM, GORM, ActiveRecord — set their database URL to the same value. To FoxByte they are ordinary Postgres clients.

4 Create · Read · Update · Delete

The full CRUD cycle — plain SQL on the left, the same operations from Node on the right.

SQL

Node.js (`pg`)

```

-- Create
INSERT INTO notes (title, body)
VALUES ('First', 'hello')
RETURNING id;

-- Read
SELECT * FROM notes ORDER BY id;

-- Update
UPDATE notes SET body = 'edited'
WHERE id = 1;

-- Delete
DELETE FROM notes WHERE id = 1;

```

```

// Create
const { rows } = await pool.query(
  `INSERT INTO notes(title, body)
  VALUES($1, $2) RETURNING *`,
  ['First', 'hello'])

// Read
await pool.query('SELECT * FROM notes')

// Update
await pool.query(
  'UPDATE notes SET body=$1 WHERE id=$2',
  ['edited', rows[0].id])

// Delete
await pool.query(
  'DELETE FROM notes WHERE id=$1',
  [rows[0].id])

```

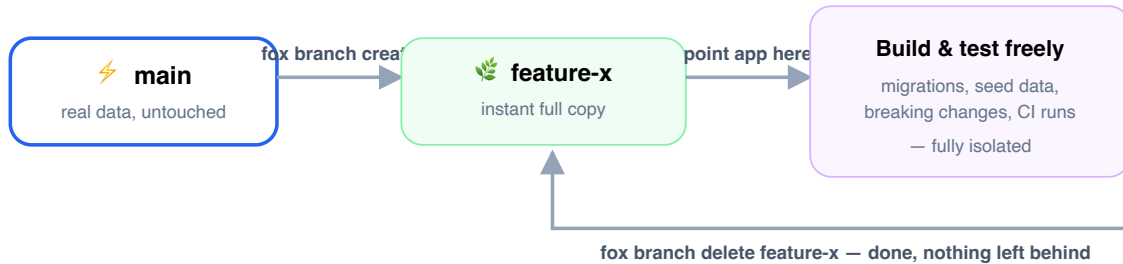
Use **parameterized queries** (\$1, \$2...) as shown — never string-concatenate user input into SQL. This is standard Postgres safety, and it works identically here.

No console handy? The in-app **SQL Console** (localhost:5173) runs these statements against any branch from the browser.

5 The superpower: a branch per feature or test

This is where FoxByte changes how you build. A branch is an **instant, isolated, full copy** of your database (copy-on-write, so it takes seconds and almost no disk). Point your app at a branch, do anything to it, then throw it away — `main` is never touched.

Branch-per-task workflow



```
fox branch create feature-x          # seconds, near-zero disk
# in your app / CI:
DATABASE_URL=postgres://dbadmin:<API_KEY>@localhost:6432/feature-x
# ...run migrations, tests, a demo, anything...
fox branch delete feature-x          # throw it away; main is untouched
```

Great for

- **Every pull request** — a preview database with production-like data.
- **CI / integration tests** — a fresh branch per run, deleted after.
- **Risky migrations** — try it on a branch first; if it breaks, delete and retry.
- **Reproducing a bug** — branch, poke at it, discard.

6 One database per AI agent

If your app runs AI agents, give each its own disposable database over HTTP — no SQL admin needed. (These HTTP endpoints require an API key; create one with `fox apikey create <email> .`)

```
curl -H "Authorization: Bearer $FOX_KEY" \
  -X POST localhost:8088/agents/alice/branch
# -> { "dsn": "postgres://...:PORT/foxbyte" } - the agent connects to that dsn
curl -H "Authorization: Bearer $FOX_KEY" \
  -X DELETE localhost:8088/agents/alice/branch # tear it down
```

Quick reference

TASK	COMMAND / VALUE
Start / stop everything	<code>fox start</code> · <code>fox stop</code>

TASK	COMMAND / VALUE
Connection string	<code>postgres://dbadmin:<API_KEY>@localhost:6432/<branch></code>
Create / list / delete a branch	<code>fox branch create list delete <name></code>
Open a SQL shell on a branch	<code>psql "...6432/<branch>"</code>
Time-travel (point-in-time restore)	<code>fox backup create</code> · <code>fox restore --to latest</code>
Web console & dashboard	<code>make web-dev</code> → <code>localhost:5173</code>

That's it. Standard PostgreSQL for your app + instant branches for your workflow. Keep this guide and the live **Docs** / **SQL Console** pages side by side as you build.